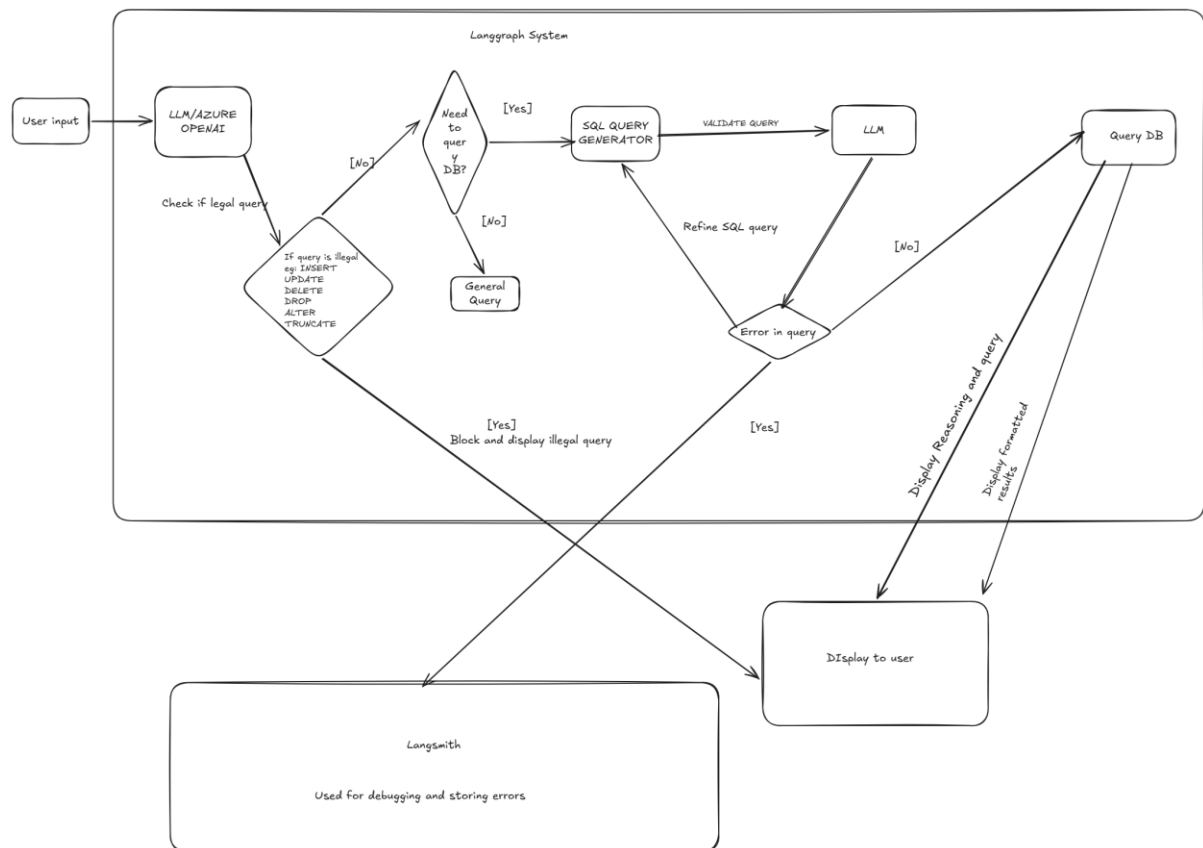


Systems UML diagram



Resources

Using this blog to understand=> Agentic Retrieval From Any SQL Database

<https://nibnab.medium.com/agentic-retrieval-from-any-sql-database-6a93024541fd>

<https://docs.langchain.com/oss/python/langchain/sql-agent>

<https://medium.com/@elsossama/building-ai-agents-that-query-sql-databases-two-practical-methods-mcp-server-langchain-00d5007d6e05>

Approach 1->Using an MCP server

We create mcp server to query db using psycopg2 or SQLAlchemy

We build a dedicated plugin for the AI (called an MCP Server) that acts as a secure middleman between the AI and our PostgreSQL database.

The Security Gate: It physically blocks dangerous commands like DROP or DELETE so the AI can only read data, never destroy it.

The Blueprint Tool: It gives the AI a clear map of our tables and columns so the AI knows exactly where things are.

The Connection Driver: It uses standard Python tools (psycopg2 or SQLAlchemy) to securely run the AI's data queries and return the answers.

Approach 2-> LangChain SQL Agent

Instead of building a separate plugin, we use an off-the-shelf toolkit from LangChain that connects the AI directly to our database.

The Pre-Built Setup: LangChain gives us the entire system ready-to-go, so we don't have to write the code for nodes, arrows, or loops from scratch.

The Smart Tools: It hands the AI four built-in tools to look at tables, inspect schemas, check the query for syntax errors, and run the search.

The Built-In Loop: If the AI writes bad SQL code, the toolkit automatically catches the database error and forces the AI to fix it and try again.